ETX Signal-X

Daily Intelligence Digest

Wednesday, March 4, 2026

10	15
ARTICLES	TECHNIQUES

Article 1

How LinkedIn Exposed Millions of Email Addresses via Voyager API

Source: securitycipher

Date: 15-Jun-2025

URL: https://osintteam.blog/how-linkedin-exposed-millions-of-email-addresses-via-voyager-api-bc09140625b9

T1 LinkedIn Voyager API URN Decoration Expansion for Unauthorized Email Disclosure

💉 Payload

```
1. Profile field injection (manual step):
   Website URL: urn:li:fs_emailAddress:8519272224

2. Decorated Voyager API request (JavaScript fetch example):

await (await fetch("https://www.linkedin.com/voyager/api/identity/dash/profiles?decoration
=%28websites*%28url~%29%29&memberIdentity=[public_identifier]&q=memberIdentity", {
  "headers": {
    "accept": "application/vnd.linkedin.normalized+json+2.1",
    "csrf-token": "ajax:[your_token_here]"
  },
  "method": "GET",
  "credentials": "include"
})).json()
```

⚔️ Attack Chain

1. Edit your own LinkedIn profile and add a new website entry with the URL set to a target email URN (e.g., `urn:li:fs_emailAddress:8519272224`).
2. Make a Voyager API request to `/voyager/api/identity/dash/profiles` with the `decoration=(websites*(url~))` parameter, your `memberIdentity`, and required headers (`accept`, `csrf-token`).
3. The API response will resolve and expand the injected URN, returning the email address associated with the referenced URN, even if it belongs to another user.
4. Extract the email address from the JSON response.

🔍 Discovery

Researcher noticed LinkedIn's Voyager API supported URN decorations and did not validate field content or access rights. Hypothesized that internal URNs could be injected into profile fields and resolved via decoration expansion. Confirmed by testing with an email URN in a website field and observing unauthorized email disclosure in the API response.

🔓 Bypass

No access control on URN resolution: Any user could expand arbitrary URNs via decoration, regardless of ownership or permissions. No validation on user-supplied profile fields: Arbitrary URNs could be injected into fields like `websites`, which would be blindly resolved by the backend.

🔗 Chain With

Enumerate email URNs (sequential or pseudo-sequential) to automate mass email extraction. Distribute attack across multiple accounts to evade rate limiting or detection. Use harvested emails for spear phishing, credential stuffing, or corporate reconnaissance.

T2 Mass Enumeration of Internal Resource URNs via Profile Field Injection

💉 Payload

```
1. Automated profile update (pseudo-code):
   For each candidate_email_urn in urn_list:
       Set website field to urn:li:fs_emailAddress:{candidate_email_urn}
       Fetch profile with decoration=(websites*(url~))
       If response contains valid email, store result

2. Example Voyager API request:
GET /voyager/api/identity/dash/profiles?decoration=(websites*(url~))&memberIdentity=[public
_identifier]&q=memberIdentity
accept: application/vnd.linkedin.normalized+json+2.1
csrf-token: ajax:[your_token_here]
Cookie: [session cookies]
```

⚔️ Attack Chain

1. Identify the URN pattern for sensitive resources (e.g., `urn:li:fs_emailAddress:{id}`) and determine if IDs are sequential or guessable.
2. Script the injection of candidate URNs into a profile field (e.g., `websites`).
3. For each injected URN, trigger a decorated Voyager API call to expand the field and resolve the resource.
4. Parse the API response for valid email addresses or other sensitive data.
5. Repeat across a large range of URN values to scrape massive datasets.

🔍 Discovery

After confirming single-instance email disclosure, researcher tested for enumeration potential by observing that email URNs were sequential/predictable. Automated the process to inject and resolve large numbers of URNs, confirming mass data extraction was feasible.

🔓 Bypass

No rate limiting per URN or per account (attack could be distributed across accounts). No backend validation of URN ownership or resource access. No monitoring for unusual decoration query patterns.

🔗 Chain With

Combine with account automation to scale attacks and evade detection. Use as a precursor for targeted attacks (phishing, ATO, social engineering) at scale.

Article 2

How I got 324$ for a PII disclosure

Source: securitycipher

Date: 05-Apr-2025

URL: https://aarnavsaboo.medium.com/324-pii-disclosure-89e6b384fab7

T1 Unauthenticated Receipt Access via Public Endpoint

💉 Payload

```
GET /buy/success?id=xxxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx HTTP/1.1
Host: prod.target.com
```

⚔️ Attack Chain

1. Enumerate subdomains for the target (e.g., using subfinder).
2. Identify live subdomains (e.g., with httpx).
3. Locate endpoints with user-controllable parameters (e.g., using paramspider or archived endpoints).
4. Access URLs of the form `/buy/success?id=<uuid>` on the discovered subdomain (e.g., prod.target.com).
5. Receive full receipt pages containing PII (full name, location, purchase details, masked payment method, email) without authentication or special headers.

🔍 Discovery

Passive recon: Subdomain enumeration revealed a production/staging server (`prod.target.com`) with no redirects or authentication. Reviewing archived endpoints and parameter discovery tools exposed `/buy/success?id=<uuid>` as a recurring pattern. Direct access yielded sensitive user data.

🔓 Bypass

No authentication, cookies, or special headers required. Endpoint is fully public; knowledge of the UUID is sufficient for access.

🔗 Chain With

If UUIDs are predictable or leaked, mass enumeration or scraping of receipts is possible. Data exposure may facilitate phishing, social engineering, or further account compromise if emails or purchase details are reused elsewhere.

Article 3

SAML Authentication Bypass Leading to Admin Panel Access

Source: securitycipher

Date: 31-Jul-2024

URL: https://medium.com/@0x_xnum/saml-authentication-bypass-leading-to-admin-panel-access-24f23812ed76

T1 SAML Signature Stripping and Attribute Manipulation for Authentication Bypass

💉 Payload

```
<?xml version="1.0" encoding="UTF-8"?>
<saml2p:AuthnRequest xmlns:saml2p="urn:oasis:names:tc:SAML:2.0:protocol" AssertionConsumerS
erviceURL="https://accounts-otp.exmaple.com/authenticated" ID="__b261eb3afa7ee41b11f2965608
172e35" IssueInstant="2024-07-30T18:12:25.595Z" ProtocolBinding="urn:oasis:names:tc:SAML:2.
0:bindings:HTTP-POST" Version="2.0">
  <saml2:Issuer xmlns:saml2="urn:oasis:names:tc:SAML:2.0:assertion">account_manager_otp_b</
saml2:Issuer>
  <!-- <ds:Signature> ... </ds:Signature> REMOVED ENTIRELY -->
  <saml2p:Extensions>
    <odin:version xmlns:odin="http://odin.vrsn.com">v3</odin:version>
  </saml2p:Extensions>
</saml2p:AuthnRequest>
```

⚔️ Attack Chain

1. Capture a legitimate SAMLResponse from the authentication flow (e.g., via login at `https://login-otp.example.com/SAML2/SSO/POST`).
2. Decode the SAMLResponse from base64 to XML.
3. Remove all `<Signature>` elements from the XML (strip the digital signature block).
4. Modify the `ID` attribute to a new unique value (e.g., using a UUID generator).
5. Change sensitive SAML attributes, such as:
6. Re-encode the manipulated SAMLResponse to base64.
7. Replace the original SAMLResponse parameter in the authentication POST request with the modified value.
8. Forward the request to the target endpoint.
9. If the service does not enforce signature validation, authentication is bypassed and attacker gains access (including admin panel if attributes are set accordingly).

🔍 Discovery

Manual inspection of SAML authentication flow and response structure. The researcher tested for signature validation weaknesses by removing the `<Signature>` element and altering SAML attributes, observing whether the service provider accepted the tampered response.

🔓 Bypass

The attack relies on the service provider failing to enforce digital signature validation on incoming SAML responses. By stripping the `<Signature>` element and modifying attributes, the attacker bypasses integrity and authenticity checks, leading to privilege escalation.

🔗 Chain With

Combine with subdomain enumeration to identify additional SAML endpoints or privileged panels. Use in conjunction with IDOR or privilege escalation bugs if attribute manipulation enables higher roles. Potential for account takeover if email or user identifier attributes are not properly validated.

T2 Automated SAML Injector Script for Mass Manipulation and Testing

💉 Payload

```
python3 script.py <OriginalSAMLResponse>
# (script at: https://github.com/0xxnum/saml_injector/blob/main/saml_injector.py)
# Automates:
# - Decoding base64 SAMLResponse
# - Removing <Signature> elements
# - Modifying attributes (OTP, Email, NameID, etc.)
# - Re-encoding to base64
```

⚔️ Attack Chain

1. Obtain the original SAMLResponse from the authentication flow.
2. Run the SAML injector script with the original SAMLResponse as input.
3. The script decodes, strips signatures, and modifies attributes as specified.
4. Script outputs a new base64-encoded SAMLResponse.
5. Replace the SAMLResponse parameter in the authentication POST request with the script's output.
6. Forward the request to the SAML endpoint.
7. If the endpoint does not validate signatures, attacker is authenticated with attacker-controlled attributes.

🔍 Discovery

Manual exploitation was automated for efficiency and scale. The script was developed to streamline the process of SAML manipulation and enable rapid testing of multiple payloads and attributes.

🔓 Bypass

Automates the signature stripping and attribute manipulation process, enabling mass testing and exploitation against SAML endpoints lacking proper validation.

🔗 Chain With

Use for fuzzing various SAML attributes to discover additional privilege escalation vectors. Integrate into automated recon workflows to identify vulnerable SAML endpoints at scale. Chain with credential stuffing or phishing to obtain initial SAMLResponses for manipulation.

Article 4

IDOR Pada Website Jual Beli Indonesia

Source: securitycipher

Date: 03-Feb-2024

URL: https://uzet.medium.com/idor-pada-website-jual-beli-indonesia-e4842bad515a

T1 IDOR Manipulation of Shipping Fee in Checkout API

💉 Payload

```
POST /api/checkout HTTP/1.1
Host: [REDACTED]
Content-Type: application/json
Cookie: sessionid=...

{
  "item_id": "12345",
  "shipping_fee": 0,
  "shipping_method": "JNE"
}
```

⚔️ Attack Chain

1. Add an item to cart and proceed to checkout.
2. Intercept the checkout request (e.g., via Burp Suite).
3. Identify the `shipping_fee` parameter in the request body.
4. Modify `shipping_fee` to an arbitrary value (e.g., `0` for free shipping or an inflated value for fraud).
5. Forward the modified request to the server.
6. Complete the payment process using the resulting QR code or payment method.

🔍 Discovery

Manual interception of the checkout process with Burp Suite, focusing on cost-related parameters after observing dynamic shipping fee calculation.

🔓 Bypass

The backend does not validate the shipping fee against the actual shipping method or item, allowing direct manipulation of the fee amount.

🔗 Chain With

Combine with coupon or discount manipulation for negative total values (potential payout/credit abuse). Use in conjunction with account-level privilege escalation to perform the attack on behalf of other users (horizontal/vertical privilege escalation).

Article 5

Discourse is an open source discussion platform. In versions prior to 3.5.4, 2025.11.2, 2025.12.1, and 2026.1.0, an endpoint lets any authenticated user bypass the ai_discover_persona access controls and gain ongoing DM access to personas that may be wire

Source: threatable

Date:

URL: https://github.com/discourse/discourse/security/advisories/GHSA-mrvm-rprq-jqqh

T1 Bypassing ai_discover_persona Access Controls for Unauthorized Persona DM Access

💉 Payload

```
POST /ai_discover/continue_conversation
Content-Type: application/json
Authorization: Bearer <attacker's valid token>

{
  "persona_id": <target_persona_id>,
  "user_id": <target_user_id>
}
```

⚔️ Attack Chain

1. Authenticate as any valid user to obtain a session or bearer token.
2. Craft a POST request to the `/ai_discover/continue_conversation` endpoint.
3. Set `persona_id` to the ID of a persona wired to restricted (e.g., staff-only) categories or sensitive RAG document sets.
4. Set `user_id` to your own user ID or any other user ID.
5. Send the request to gain ongoing DM access to the persona, bypassing intended access controls.

🔍 Discovery

Review of the AI Discover controller code revealed that access controls for `ai_discover_persona` were not enforced on the `continue_conversation` endpoint, and that arbitrary `user_id` values were accepted. This was likely identified during code review or fuzzing endpoints with crafted parameters.

🔓 Bypass

The endpoint does not validate that the authenticated user is authorized for the specified `persona_id`, nor does it restrict `user_id` to the caller's own account, allowing full bypass of persona and user access controls.

🔗 Chain With

Use with personas wired to staff-only categories to exfiltrate sensitive staff discussions or documents. Trigger AI conversations as other users (impersonation), potentially generating confusing or abusive PM traffic or manipulating audit trails.

T2 User Impersonation via Arbitrary user_id Parameter in AI Conversation Trigger

💉 Payload

```
POST /ai_discover/continue_conversation
Content-Type: application/json
Authorization: Bearer <attacker's valid token>

{
  "persona_id": <any_persona_id>,
  "user_id": <victim_user_id>
}
```

⚔️ Attack Chain

1. Authenticate as any valid user to obtain a session or bearer token.
2. Craft a POST request to `/ai_discover/continue_conversation`.
3. Set `user_id` to the ID of a victim user (not your own).
4. Set `persona_id` to any persona (including those with access to sensitive or automated tooling).
5. Send the request to trigger an AI conversation as the victim, causing unwanted or malicious PM traffic attributed to the victim.

🔍 Discovery

Analysis of the endpoint parameters revealed that `user_id` was not restricted to the authenticated user's ID. This was likely discovered by testing the endpoint with different `user_id` values and observing the resulting behavior in the DM system.

🔓 Bypass

No server-side check enforces that `user_id` matches the authenticated user, enabling arbitrary user impersonation for AI-triggered DMs.

🔗 Chain With

Combine with social engineering or spam to flood users with unwanted AI-generated messages attributed to other users. Abuse to poison audit logs or trigger automated workflows tied to user DMs.

Article 6

Broken Access Control leads to Take Admin Role

Source: securitycipher

Date: 13-May-2024

URL: https://0xebn-taimia.medium.com/broken-access-control-leads-to-take-admin-role-920e46d496b9

T1 Privilege Escalation via Modifiable "type" Parameter in Account Creation

💉 Payload

```
POST /audience/login HTTP/1.1
Host: mktgadt.example.com
Content-Type: application/x-www-form-urlencoded

username=testuser&password=TestPass123&type=admin
```

⚔️ Attack Chain

1. Discover subdomain `mktgadt.example.com` (returns 404, content-length 0).
2. Fuzz endpoints with ffuf, identify `/audience/login` (returns 200 OK, admin login page with account creation option).
3. Register a new account; observe `type` parameter in the request defaults to `viewer`.
4. Modify the `type` parameter value to `admin` (or `super-admin`, `editor`).
5. Submit the request and gain elevated privileges (admin/super-admin/editor access).
6. Access admin/super-admin interface and functionalities.

🔍 Discovery

Subdomain enumeration (subfinder) revealed a dead subdomain. Fuzzing with ffuf on the dead subdomain uncovered a hidden admin login endpoint. Manual inspection of account creation request in Burp Suite revealed the modifiable `type` parameter.

🔓 Bypass

No server-side validation of the `type` parameter during account creation; direct privilege escalation by supplying arbitrary role values.

🔗 Chain With

Use super-admin access to edit or escalate privileges of other accounts. Potential for lateral movement or persistence by creating multiple privileged accounts.

T2 Role Enumeration and Arbitrary Role Assignment

💉 Payload

```
POST /audience/login HTTP/1.1
Host: mktgadt.example.com
Content-Type: application/x-www-form-urlencoded

username=eviluser&password=EvilPass123&type=super-admin
```

⚔️ Attack Chain

1. Register multiple accounts, each time setting the `type` parameter to a different value (`super-admin`, `admin`, `editor`, `viewer`).
2. Confirm that each role grants corresponding interface access and privileges (e.g., super-admin can edit/delete/change privileges, admin can delete, editor can edit activity, viewer can only view).

🔍 Discovery

Observed the existence of multiple role types in the application logic and tested each by modifying the `type` parameter.

🔓 Bypass

No restrictions on accepted values for the `type` parameter; any valid role string is accepted and applied.

🔗 Chain With

Use super-admin to escalate privileges of other accounts or to create a network of privileged users for persistence.

Article 7

A security vulnerability has been detected in itsourcecode Student Management System 1.0. This issue affects some unknown processing of the file /enrollment/index.php. Such manipulation of the argument ID leads to sql injection. It is possible to launch t

Source: threatable
Date:
URL: https://github.com/ltranquility/CVE/issues/34

T1 Boolean-based Blind SQL Injection via 'id' Parameter

💉 Payload

```
GET /ramonsys/enrollment/index.php?view=viewsced&id=20004623 AND 9733=9733 HTTP/1.1
Host: <target>
```

⚔️ Attack Chain

1. Send a GET request to `/ramonsys/enrollment/index.php` with the `view=viewsced` parameter and the `id` parameter set to a valid value followed by `AND 9733=9733`.
2. Observe the application's response for behavioral differences indicating true/false evaluation (e.g., page content changes or error suppression).

🔍 Discovery

Manual parameter fuzzing of the `id` parameter with boolean logic operators; triggered by lack of input sanitization and unauthenticated access.

🔓 Bypass

No authentication required; direct exploitation of GET parameter.

🔗 Chain With

Can be chained with time-based or UNION-based SQLi for data extraction or privilege escalation.

T2 Time-based Blind SQL Injection via 'id' Parameter

💉 Payload

```
GET /ramonsys/enrollment/index.php?view=viewsced&id=20004623 AND (SELECT 8104 FROM (SELECT
(SLEEP(5)))UOOa) HTTP/1.1
Host: <target>
```

⚔️ Attack Chain

1. Send a GET request to `/ramonsys/enrollment/index.php` with `view=viewsced` and `id=20004623 AND (SELECT 8104 FROM (SELECT(SLEEP(5)))UOOa)`.
2. Measure response time; a 5-second delay indicates successful injection and code execution.

🔍 Discovery

Time-based payloads used to confirm SQL injection when no visible error or output is present; identified via sqlmap and manual testing.

🔓 Bypass

No authentication required; works even when output is suppressed.

🔗 Chain With

Can be used to enumerate database structure and extract data via time-based inference.

T3 UNION-based SQL Injection Data Extraction via 'id' Parameter

💉 Payload

```
GET /ramonsys/enrollment/index.php?view=viewsced&id=-4747 UNION ALL SELECT NULL,NULL,CONCAT
(0x7178626271,0x75627158516271485867635 34f51436968644f71796c656a66775857705851745154736957415442,0x71717a6271),NULL,NULL,NULL,NULL,NULL,NULL,NULL-- - HTTP/1.1
Host: <target>
```

⚔️ Attack Chain

1. Send a GET request to `/ramonsys/enrollment/index.php` with `view=viewsced` and a negative `id` value followed by a UNION ALL SELECT statement with 10 columns.
2. Use `CONCAT` and hex-encoded string markers to identify successful injection and locate output in the response.
3. Adjust the number of NULLs to match the column count if necessary.

🔍 Discovery

Column count and output location determined via sqlmap and manual UNION payload crafting; hex markers used for output identification.

🔓 Bypass

No authentication required; works if output is reflected in the page.

🔗 Chain With

Direct data extraction (usernames, hashes, etc.); can be chained with privilege escalation or lateral movement if database credentials are reused.

Article 8

“Day 25: The Cloud Heist — How a Forgotten Webhook Tester Gave Me the Keys to AWS”

Source: securitycipher

Date: 30-Aug-2025

URL: https://infosecwriteups.com/day-25-the-cloud-heist-how-a-forgotten-webhook-tester-gave-me-the-keys-to-aws-0e2876b515a8

⚠️ No actionable techniques found

Article 9

Bypassed Access Control Using a Simple Method-Based Request Trick — Access Control Labs

Source: securitycipher

Date: 24-Jan-2025

URL: https://bashoverflow.medium.com/bypassed-access-control-using-a-simple-method-based-request-trick-access-control-labs-61ec15242d88

T1 HTTP Method-Based Access Control Bypass

💉 Payload

```
GET /admin-roles?username=wiener&action=upgrade HTTP/1.1
Host: <target>
Cookie: session=<wiener-session-cookie>
```

⚔️ Attack Chain

1. Log in as a low-privileged user (e.g., `wiener:peter`).
2. Attempt to escalate privileges by sending the standard POST request to `/admin-roles` with body `username=wiener&action=upgrade` (results in 401 Unauthorized).
3. Change the HTTP method from POST to GET and move the parameters to the query string: `/admin-roles?username=wiener&action=upgrade`.
4. Send the modified GET request with the low-privilege user's session cookie.
5. Observe that the request is now successful and the user's privileges are escalated.
6. Confirm in the application that the privilege escalation took effect.

🔍 Discovery

Observed that POST requests for privilege changes were blocked for non-admins (401 Unauthorized). Hypothesized that access control was method-dependent. Tried changing the request method to GET and moving parameters to the query string, which bypassed the restriction.

🔓 Bypass

The server only enforced access control on POST requests but failed to apply the same checks to GET requests. By switching the method, the endpoint's logic was bypassed.

🔗 Chain With

Can be chained with session fixation or low-privilege account compromise for privilege escalation. May allow lateral movement if other endpoints are similarly protected only by method-based controls.

Article 10

Unauthorized Access to Internal Panel via Response Manipulation

Source: securitycipher

Date: 19-Feb-2025

URL: https://0onoproblem.medium.com/unauthorized-access-to-internal-panel-via-response-manipulation-0b6baaae9346

T1 Authentication Bypass via Client-Side Response Status Manipulation

💉 Payload

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: ...

{"success":false,"message":"Invalid credentials"}
```

⚔️ Attack Chain

1. Navigate to the target login page (e.g., https://example.com).
2. Enter any random username and password into the login form.
3. Intercept the login request/response using a proxy tool (e.g., Burp Suite).
4. Observe that the server responds with a 401 Unauthorized status code.
5. Modify the intercepted response, changing the status code from 401 to 200 OK.
6. Forward the modified response to the client.
7. The client application treats the authentication as successful and grants access to the internal panel.

🔍 Discovery

Manual testing of authentication flows with a proxy; noticed that client logic relied on HTTP status code rather than actual authentication result in the response body.

🔓 Bypass

By altering the HTTP response status code from 401 to 200, the client-side logic is tricked into assuming authentication succeeded, despite the server's intention to deny access.

🔗 Chain With

Can be chained with client-side session fixation or privilege escalation if the panel exposes further sensitive endpoints post-login. Useful for lateral movement if the internal panel exposes additional functionality or weak session controls.

Automated with ❤️ by ethicxlhuman